



MCA Semester – IV Final Report

Name	Chethan bv
USN	241VMTR01537
Elective	Full Stack Development
Date of Submission	09/08/2026



NovaBank: A Secure Full-Stack Digital Banking Web Application with AI-Driven Fraud Detection and Financial Health Analytics

Research Project submitted to Jain Online (Deemed-to-be University)

In partial fulfilment of the requirements for the award of:

Master of Computer Applications

Submitted by

Chethan bv

USN

241VMTR01537

*Under the
guidance of*

Chandra Bhanu

DECLARATION

*I, **Chethan bv**, hereby declare that this Project Report has been prepared by me under the guidance of **Chandra Bhanu**. I declare that this Project is towards the partial fulfilment of the credit requirement for the course “Capstone Project,” which is part of the Master of Computer Applications degree given by Jain University, Bengaluru. I declare that the work done by me towards this Project is original in nature and is my own contribution.*

Place: Gonnikoppal

Date: 09/08/2026

Chethan bv

USN: 241VMTR01537

ACKNOWLEDGEMENT

I express my sincere gratitude to Jain Online (Deemed-to-be University) for providing the academic framework and learning environment required to complete this capstone project. The Master of Computer Applications programme has provided valuable exposure to frontend engineering, backend architecture, database design, software security, and project execution.

I am deeply thankful to Chandra Bhanu for the guidance, feedback, and encouragement provided throughout the development of this project. The support received during the design, implementation, debugging, validation, and documentation phases was instrumental in shaping NovaBank into a complete and presentable digital banking solution.

I also acknowledge the contribution of the course structure, learning resources, and practical assignments that helped build the technical foundation required to implement the application.

Finally, I would like to thank my family and well-wishers for their support and motivation throughout the successful completion of this capstone project.

EXECUTIVE SUMMARY

Nova Bank is a full-stack digital banking web application being developed as a capstone project in Full Stack Development. The project focuses on delivering a secure, role-based banking platform where customers can register, log in, manage beneficiaries, view account details, and perform protected banking operations, while administrators can monitor customer information and platform-wide activity through a dedicated dashboard.

The project uses React, Tailwind CSS, Axios, and React Router on the frontend, with Java Spring Boot, Spring Security, JWT authentication, Spring Data JPA, and MySQL on the backend. During the interim phase, the core authentication flow has been implemented, beneficiary management is operational, and the admin module can retrieve customer details without exposing sensitive fields such as passwords. Additional modules such as secure transfer processing, fraud analytics, transaction history, and financial health insights have also been scaffolded and largely implemented, placing the project ahead of the minimum interim requirement baseline.

This interim report documents the current project scope, methodology, architecture, progress achieved, remaining tasks, and the planned timeline for completion. It also provides evidence that the mandatory interim submission features are functional and aligned with the capstone rubric.

TABLE OF CONTENTS

Title	Page Nos.
Executive Summary	4
Introduction	6
Project Scope and Objectives	7
Methodology	8
System Architecture and Design	9
Progress and Accomplishments	13
Future Work and Timeline	16
Conclusion	17

1. INTRODUCTION

The banking domain has rapidly evolved from branch-centric operations to digital-first customer service models. Users now expect seamless access to banking functions such as account management, fund transfer, transaction visibility, and fraud monitoring through secure web-based systems. In response to these expectations, NovaBank is proposed as a secure full-stack digital banking application that demonstrates how modern frontend and backend technologies can be integrated to deliver a production-style banking workflow.

The primary goal of the project is to build a role-based application supporting two actors: Customer and Admin. Customers should be able to register, log in, manage beneficiary payees, access account information, and use banking services through an intuitive interface. Admin users should be able to monitor customer details and platform activity without accessing sensitive information such as passwords. The project is significant because it combines secure authentication, domain modeling, REST API design, relational database implementation, and responsive user interface development within one coherent full-stack system.

From a Full Stack Development perspective, the project is relevant because it demonstrates the complete path from requirement analysis and database design to frontend integration and API security. It also introduces applied concerns such as stateless JWT authentication, layered architecture, DTO-based API design, transactional consistency, and analytics-oriented service modules.

2. PROJECT SCOPE AND OBJECTIVES

The scope of the interim phase is focused on establishing the secure foundation of the banking platform and implementing the minimum functional requirements mandated for evaluation. The report therefore concentrates on the authentication workflow, beneficiary management, and admin-side customer visibility, while also documenting the broader architecture that supports later modules.

Within the current scope, the project covers customer registration and login, JWT-based session handling, role-protected routing, customer beneficiary creation and retrieval, and admin access to customer details via safe DTO responses. The broader project architecture also includes account summary, transaction history, fund transfer, fraud analytics, and financial insight modules, although the interim assessment emphasizes the earlier milestone features.

The main scope boundary is that the current project remains a web application and does not include native mobile apps, inter-bank payment gateway integration, or a machine-learning-based fraud engine. The current implementation of beneficiary management stores beneficiary name, bank name, account number, IFSC code, and nickname. A configurable per-beneficiary max limit can be treated as a planned refinement for final-phase alignment with extended banking specifications.

Project Objectives

- Implement a secure customer registration and login workflow using Spring Security, BCrypt, JWT, and React-based authentication state handling.
- Allow customers to add and manage beneficiaries with core payee details required for future banking transactions.
- Allow administrators to view customer details through protected APIs without exposing sensitive fields such as passwords.
- Establish a scalable layered backend architecture using controller, service, repository, dto, entity, config, security, and exception packages.
- Prepare the platform for advanced modules such as transfer processing, transaction history, fraud analytics, and financial health insights in subsequent milestones.

3. SYSTEM ARCHITECTURE

NovaBank follows a client-server architecture in which the React frontend communicates with a Spring Boot REST backend over HTTP. The backend uses a layered design to separate concerns across controller, service, repository, dto, and security layers. This design improves maintainability and helps ensure that sensitive business logic, such as authentication and transfer processing, is not mixed with presentation concerns.

The chosen architectural pattern is a modular monolith with enterprise-style layering. This is appropriate for a capstone project because it keeps deployment and reasoning manageable while still demonstrating secure, scalable, and professional software structure. JWT-based stateless authentication is used so that the frontend can call protected backend APIs without relying on server-side sessions.

Key design principles used in the project include separation of concerns, principle of least privilege, DTO-based response shaping, stateless authentication, atomic database transactions for financial operations, and reusable component design on the frontend.

Figure 1. Context Diagram of NovaBank

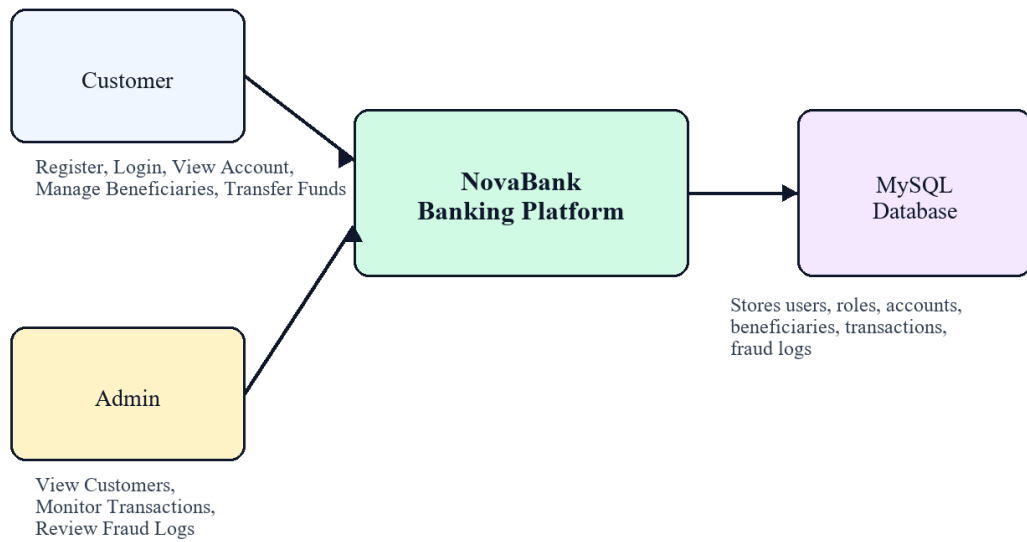


Figure 2. Layered System Architecture

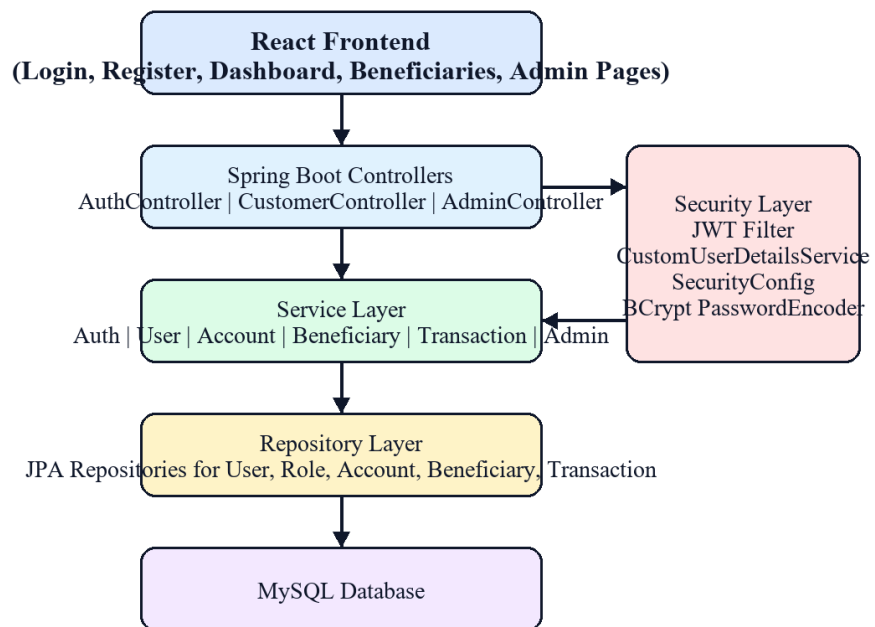


Figure 3. Interim Use Case Diagram

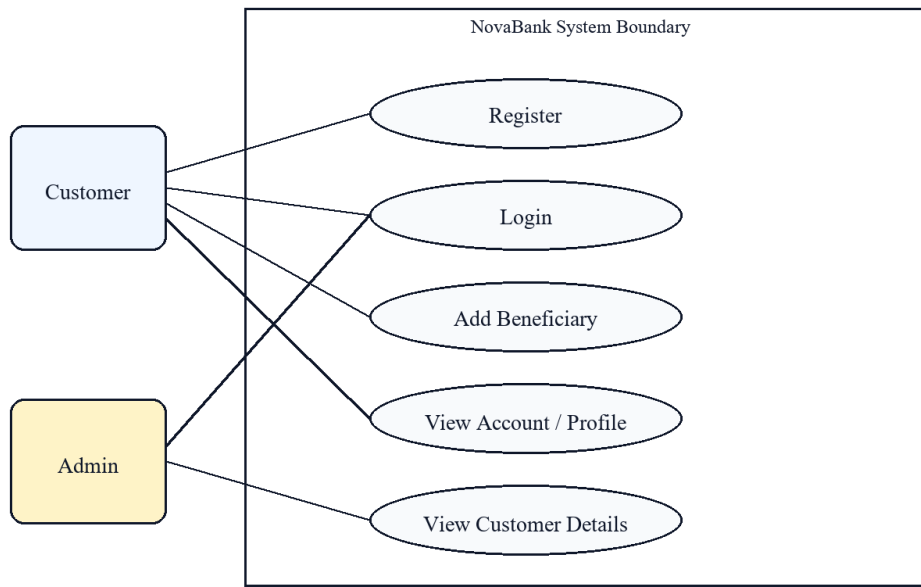
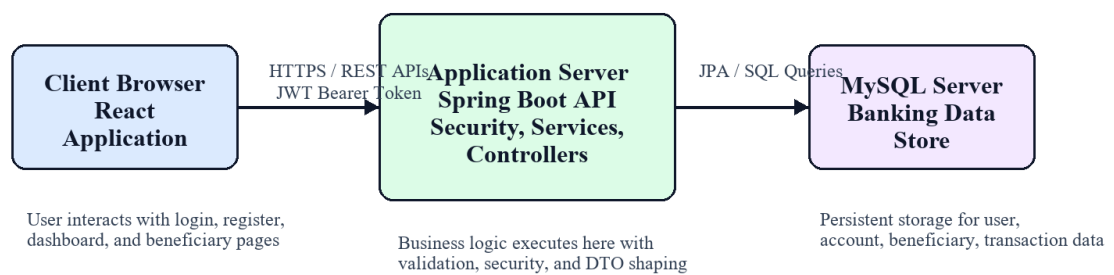


Figure 7. Deployment / Runtime View



4. TECHNOLOGIES USED

The project uses a technology stack selected to balance developer productivity, enterprise relevance, security, maintainability, and support for structured relational data.

Layer	Technologies / Tools
Frontend	React 18, Vite, Tailwind CSS, Axios, React
Backend	Java 17, Spring Boot 3, Spring Security, JWT, Spring Data JPA, Maven
Database	MySQL 8.0, Hibernate ORM
Documentation and API Inspection	Swagger UI via SpringDoc OpenAPI, Postman
Build and Packaging	Vite production build and Maven package lifecycle
Design Approach	Layered client-server architecture using a modular monolith

Technology Selection Rationale

Technology	Reason for Use
React + Vite	Fast SPA development, routing flexibility, and reusable component design.
Tailwind CSS	Consistent utility-driven styling for structured enterprise UI composition.
Spring Boot	Mature Java backend framework suitable for enterprise-style layered applications.
Spring Security + JWT	Secure stateless authentication and role-based authorization.
Spring Data JPA	Simplifies repository development over normalized relational entities.
MySQL	Supports structured data, referential integrity, joins, and transactional consistency.
Swagger / OpenAPI	Supports API inspection, contract verification, and manual testing.

5. FRONT-END DEVELOPMENT

The frontend is engineered as a responsive Single Page Application using React 18. Tailwind CSS provides styling, React Router handles navigation and protected routes, Axios manages API communication, and Chart.js supports financial data visualization.

Reusable components are organized around an application shell, authentication context, protected routes, data tables, form inputs, statistical cards, and risk indicators. ProtectedRoute prevents unauthenticated navigation to secured views while backend authorization remains the final enforcement layer.

The final interface was refined into a realistic enterprise banking portal. The design uses compact spacing, structured navigation, dense operational tables, clear labels, restrained controls, and information-focused dashboards instead of oversized cards, excessive gradients, or marketing-style visuals.

Table - Frontend Pages and Functionalities

Page	Purpose
Login	Authenticates customer or administrator users and routes them to the appropriate dashboard.
Register	Creates a customer profile and provisions access credentials and a banking account.
Account Overview	Displays balance, account information, notices, and recent transactions.
Payments & Transfers	Submits secured transfer requests using saved beneficiaries.
Beneficiary Management	Creates, updates, views, and removes reusable transfer beneficiaries.
Transaction History	Provides searchable debit and credit ledger visibility.
Fraud Review	Displays rule-based fraud scores, risk levels, and evaluation reasons.
Financial Insights	Shows health score, cash-flow trends, savings rate, and suggestions.
Profile & Contact	Maintains customer name, phone number, and registered address.
Operations Dashboard	Allows administrators to monitor customers, transactions, KPIs, and fraud alerts.

6. BACK-END DEVELOPMENT

The backend is implemented as a modular Spring Boot REST API with packages for controllers, services, repositories, DTOs, entities, configuration, security, and exception handling. This separation isolates HTTP concerns, business rules, persistence logic, and security configuration.

Public authentication endpoints support registration and login. Customer services provide profile management, account summary retrieval, beneficiary CRUD, transaction listing and search, transfer execution, fraud analytics, and financial insight retrieval. Administrator services provide dashboard metrics, customer listing, transaction review, and fraud-log access.

5.1 Transfer Processing

The transfer service loads the sender account, validates beneficiary ownership, locks update-sensitive account records, verifies account status and sufficient balance, and updates both balances within a transactional boundary. The operation persists a sender-side debit record and receiver-side credit record under the same reference number. Fraud evaluation is stored separately in the fraud log.

5.2 Backend Modules

Module	Description
Authentication	Registration, credential verification, password hashing, and JWT issuance.
Customer	Profiles, account summaries, transaction history, and customer financial data.
Admin	Platform KPIs, customer review, transaction audit, and fraud visibility.
Transfer	Secure transfers, atomic balance updates, and ledger records.
Fraud Engine	Transfer-risk evaluation, scoring, classification, and fraud-log persistence.
Financial Insights	Credit/debit analysis, savings rate, health score, and suggestions.
Security	JWT filtering, endpoint authorization, CORS, and BCrypt password hashing.

5.3 Core REST APIs

Table. Core REST API Catalogue

Method	Endpoint	Purpose
POST	/api/auth/register	Registers a customer and provisions a banking account.
POST	/api/auth/login	Authenticates credentials and returns a JWT response.
GET	/api/customer/profile	Fetches the authenticated customer's profile.
PUT	/api/customer/profile	Updates customer profile details.
GET	/api/customer/account	Fetches account number, balance, currency, and status.
GET	/api/customer/beneficiaries	Lists customer-owned beneficiaries.
POST	/api/customer/beneficiaries	Creates a beneficiary.
PUT	/api/customer/beneficiaries/{id}	Updates a beneficiary.
DELETE	/api/customer/beneficiaries/{id}	Removes a beneficiary.
GET	/api/customer/transactions	Lists customer transactions with search.
POST	/api/customer/transfer	Executes an atomic account-to-account transfer.
GET	/api/customer/fraud-analytics	Returns customer fraud analysis history.
GET	/api/customer/financial-insights	Returns health score, trends, and suggestions.
GET	/api/admin/dashboard	Returns aggregate admin dashboard statistics.
GET	/api/admin/customers	Lists customer details without exposing password data.
GET	/api/admin/transactions	Lists transactions for administrative review.
GET	/api/admin/fraud-logs	Lists fraud log entries.

7. DATABASE DESIGN

NovaBank uses MySQL as its relational database management system. This choice is appropriate because the banking domain depends on strongly structured data, referential integrity, predictable joins, and transactional consistency. The schema is normalized across users, roles, accounts, beneficiaries, transactions, and fraud logs to reduce redundancy and simplify rule enforcement.

The users table stores identity and contact information, while roles and user_roles implement normalized authorization mapping. The accounts table is separated from users so account-specific attributes such as account number, balance, currency, and status can evolve independently. The beneficiaries table allows reusable payee management for each customer. The transactions table acts as an immutable ledger, and fraud_logs stores rule-based risk analysis linked to transfer activity.

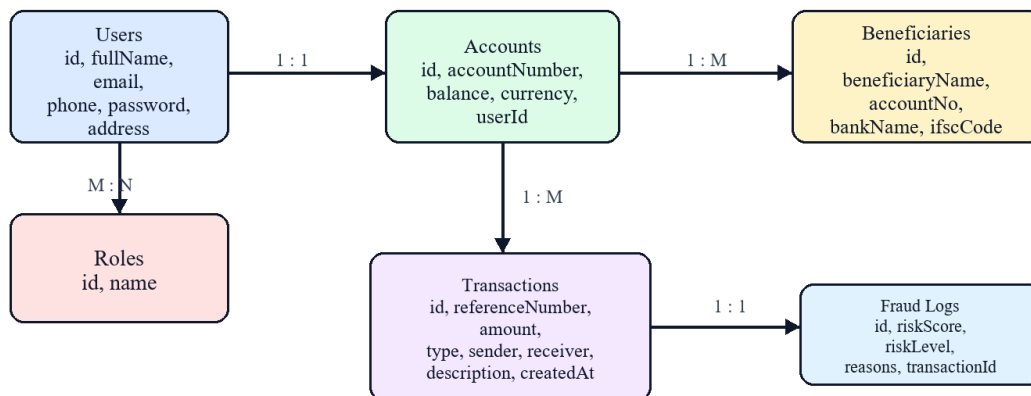
Each transfer can create both a sender-side debit row and a receiver-side credit row under a shared reference number. This design improves traceability, customer visibility, and administrative auditability.

Table . Database Tables and Purpose

Table	Purpose
roles	Stores role definitions such as ROLE_ADMIN and ROLE_CUSTOMER.
users	Stores customer/admin identity, contact details, password hash, and status.
user_roles	Maintains normalized many-to-many mapping between users and roles.
accounts	Stores account number, balance, currency, active flag, and linked user.
beneficiaries	Stores beneficiary nickname, payee identity, bank details, and owner.
transactions	Stores immutable debit/credit ledger records with sender/receiver linkage.
fraud_logs	Stores evaluated risk score, risk level, amount, and reasons for transfers.

Table . Normalization and Design Justification

Design Choice	Justification
User and role split	Avoids repeated role text in user records and supports flexible role assignment.
Account separated from user	Keeps financial attributes independent from identity attributes.
Beneficiary table per customer	Enables reusable payees without duplicating transfer data.
Ledger-style transactions	Retains an auditable history rather than mutating past transfer records.
Fraud log separation	Allows monitoring logic to evolve without contaminating transaction core data.

Figure 4. Database Entity Relationship Diagram of NovaBank**Figure 4. Database Entity Relationship View**

8.AUTHENTICATION AND SECURITY

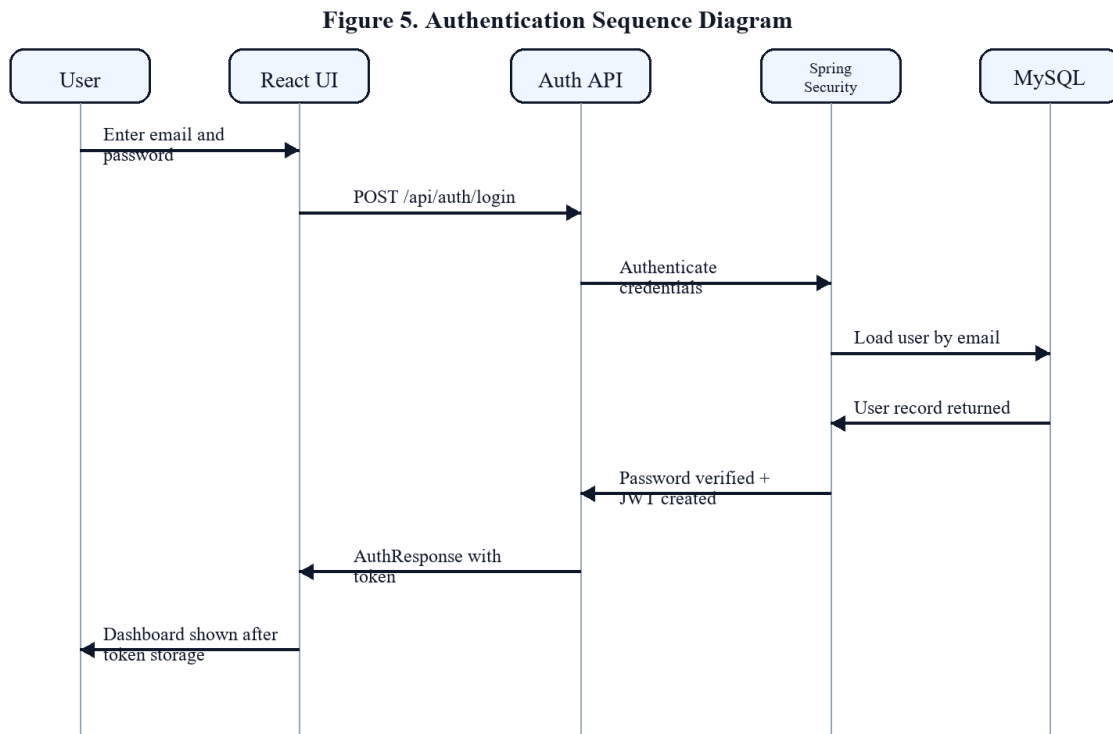
Authentication and authorization are enforced using Spring Security and JWT. Public access is restricted to authentication and documentation endpoints, while customer routes and administrator routes are protected by role-based rules.

During login, the backend authenticates submitted credentials using Spring Security and BCrypt. On successful authentication, a signed JWT carrying the authenticated identity and roles is returned. The frontend stores the token and an Axios interceptor adds it to the Authorization header for subsequent protected requests. JwtAuthenticationFilter validates the bearer token and populates the Spring Security context.

Security extends beyond login. Sensitive entity data such as password hashes is not exposed through API responses because DTOs are used consistently. Transfer execution includes active-account checks, beneficiary ownership validation, and transactional update boundaries. Fraud analysis is stored separately from the transaction ledger so that monitoring remains modular and auditable.

Table - Security Controls Implemented

Control	Purpose
BCrypt password encoding	Prevents storage of plain-text passwords in the database.
JWT stateless authentication	Enables protected API access without server-side session dependency.
Role-based authorization	Restricts administrator functions and protects customer APIs.
DTO-based responses	Avoids exposing entity internals such as password hashes.
Transactional transfer service	Protects money movement with atomic balance updates and consistent ledger writes.
Beneficiary ownership validation	Prevents transfers to unauthorized beneficiary IDs.
CORS configuration	Restricts browser requests to configured frontend origins.
Input validation	Rejects malformed or invalid request data.

Figure 5. JWT Authentication Sequence Diagram

9. PROJECT MANAGEMENT

The capstone was developed using an iterative and milestone-driven approach similar to Agile practice. Work was divided into requirements analysis, system design, backend foundation, security implementation, customer modules, administrator modules, transaction refinement, analytics, frontend redesign, validation, and report preparation.

The implementation was dependency-aware. Authentication was stabilized before protected customer operations were built; beneficiary management preceded meaningful transfer testing; transfer correctness was hardened before fraud and financial insight modules were evaluated; and the frontend was refined after the core workflows became stable.

Table . Development Phases

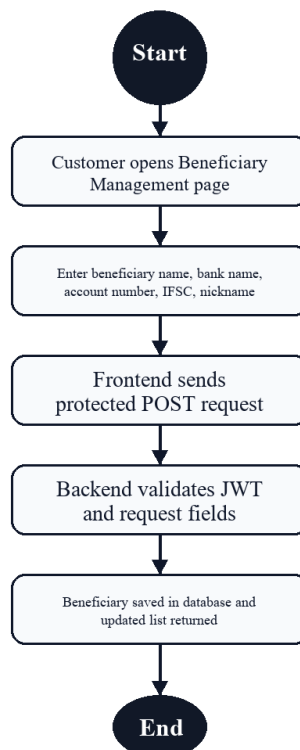
Phase	Outcome
Phase 1	Requirements, problem definition, user roles, database schema, and package planning.
Phase 2	Registration, login, BCrypt password handling, JWT issuance, and validation.
Phase 3	Customer profile, account summary, beneficiary CRUD, and transaction listing.
Phase 4	Admin dashboard metrics, customer review, transaction review, and fraud visibility.
Phase 5	Transactional money transfer, fraud scoring, and financial insight logic.
Phase 6	Enterprise frontend redesign, build verification, validation, and final documentation.

Risk Assessment and Mitigation Matrix.

Risk Area	Possible Impact	Mitigation Strategy
Payload mismatch	Differences between React models and Spring Boot DTOs can cause API failures.	Document API contracts using OpenAPI/Swagger and revalidate key screens after backend updates.
Concurrency conflicts	Concurrent transfers can cause race conditions and balance inaccuracy.	Use pessimistic write locking on account queries to serialize balance updates.
Token hijacking	Intercepted JWTs may allow unauthorized access.	Use token expiry limits, HTTPS transport, and server-side token validation.
Scope creep	Extended requirements can delay target completion.	Freeze the core feature set and focus on robust internal transfers and simulated integrations.

Figure 6. Beneficiary Management Activity Diagram

Figure 6. Beneficiary Management Activity Diagram



10.RESULTS AND EVALUATION

NovaBank has been implemented against the functional and security goals defined for the capstone. Core endpoints covering registration, profile update, account access, beneficiary management, transfer execution, fraud auditing, and financial analytics are integrated with the React frontend and return structured API responses.

Transaction safety was evaluated by considering concurrent transfers involving the same account. Without database locking, concurrent balance updates can produce race conditions. Integrating Spring Data JPA pessimistic write locking on the account lookup query serializes the relevant database operations, reducing the risk of double-spending and inconsistent balances.

The application also benefits from stateless JWT authentication, indexed relational data, reusable React components, and separated service-layer responsibilities. These design decisions improve maintainability and support predictable behaviour as the system grows.

Table . Capstone Functionality Implementation Coverage

Functionality	Status	Implementation Detail
Authentication Flow	Fully Implemented	User registration, BCrypt password hashing, JWT generation, and role guards.
Account Provisioning	Fully Implemented	New customers receive a checking account with a unique account number and balance.
Beneficiary CRUD	Fully Implemented	Customers can add, edit, view, and delete saved payee records.
Atomic Transfers	Fully Implemented	Pessimistic locking protects concurrent balance updates within one transaction.
Fraud Engine	Fully Implemented	Transactions are classified into LOW, MEDIUM, or HIGH risk using rule-based conditions and logged.
Financial Insights	Fully Implemented	Monthly credit/debit totals, savings rate, health score, and suggestions are calculated.
Admin Dashboard	Fully Implemented	Displays system-wide statistics, transactions, and fraud alerts.
Profile Management	Fully Implemented	Customers can retrieve and update name, address, and phone details.

Table . Evaluation Summary

Dimension	Evaluation
Functional completeness	Core customer and administrator workflows are implemented end to end.
Security posture	JWT, BCrypt, role-based authorization, DTO isolation, validation, and ownership checks provide a strong academic security baseline.
Data integrity	Transactional transfer logic, debit/credit ledger recording, and normalized design improve financial correctness and auditability.
User interface quality	The final frontend is designed as an enterprise banking portal with task-oriented screens and operational tables.
Maintainability	Layered packages and DTO-based contracts separate presentation, business, persistence, and security concerns.
Limitations	A complete automated test suite, refresh-token lifecycle, and production-grade notification workflows remain future enhancements.

11. CONCLUSION

The NovaBank project demonstrates the successful design and implementation of a modern, secure full-stack banking portal. The project integrates core web standards, Spring Security controls, database synchronization safeguards, and rule-based data analytics into a functional Single Page Application.

The project shows the importance of DTO design in preventing sensitive data exposure, pessimistic locking in protecting concurrent financial operations, and structured Agile planning in delivering multiple interconnected deliverables. The layered architecture also keeps the application maintainable and easier to extend.

The completed application satisfies the main project requirements by providing secure customer and administrator workflows, protected fund transfers, beneficiary management, searchable transaction history, fraud monitoring, and financial health analytics. Future enhancements may include OAuth2 or federated authentication, high-risk transaction notifications, machine-learning-based fraud scoring, stronger token lifecycle management, automated unit and integration testing, and Docker/cloud deployment.